

BÁO CÁO CODE GIẢI PHƯƠNG TRÌNH $f(x)=0$

Môn: Vật lý tính toán

Computational Physics

1 Các phương pháp giải số

Để giải gần đúng phương trình phi tuyến $f(x) = 0$ chúng ta có nhiều phương pháp số khác nhau, gồm:

- Phương pháp chia đôi khoảng cách (Bisection),
- Phương pháp điểm cố định (Fixed-point iteration),
- Phương pháp Newton–Raphson,
- Phương pháp dây cung (Secant).

2 Các hàm số được khảo sát

Trong chương trình, em chọn ba hàm số sau để sử dụng để kiểm tra gồm:

$$f_1(x) = e^{-x} - x^2 = 0,$$

$$f_2(x) = x^3 - \cos x = 0,$$

$$f_3(x) = \ln(x+2) + \cosh x - 3 = 0.$$

Đạo hàm của các hàm trên lần lượt là:

$$f'_1(x) = -e^{-x} - 2x,$$

$$f'_2(x) = 3x^2 + \sin x,$$

$$f'_3(x) = \frac{1}{x+2} + \sinh x.$$

2.1 Biến đổi về dạng $x = g(x)$ dành cho phương pháp điểm cố định

Để áp dụng phương pháp điểm cố định, ta biến đổi mỗi phương trình đã cho về dạng tương đương

$$x = g(x).$$

Với phương trình thứ nhất:

$$f_1(x) = e^{-x} - x^2 = 0$$

$$\Rightarrow e^{-x} = x^2$$

$$\Rightarrow x = \sqrt{e^{-x}} = e^{-x/2}.$$

Do đó chọn

$$g_1(x) = e^{-x/2}.$$

Với phương trình thứ hai:

$$f_2(x) = x^3 - \cos x = 0$$

$$\Rightarrow x^3 = \cos x$$

$$\Rightarrow x = \sqrt[3]{\cos x}.$$

Do đó chọn

$$g_2(x) = \sqrt[3]{\cos x}.$$

Với phương trình thứ ba:

$$\begin{aligned} f_3(x) &= \ln(x+2) + \cosh x - 3 = 0 \\ \Rightarrow \cosh x &= 3 - \ln(x+2) \\ \Rightarrow x &= \cosh^{-1}(3 - \ln(x+2)) \end{aligned}$$

Do đó chọn

$$g_3(x) = \cosh^{-1}(3 - \ln(x+2)).$$

Như vậy, công thức lặp điểm cố định cho ba bài toán lần lượt là

$$\begin{aligned} x_{n+1} &= g_1(x_n) = e^{-x_n/2}, \\ x_{n+1} &= g_2(x_n) = \sqrt[3]{\cos x_n}, \\ x_{n+1} &= g_3(x_n) = \cosh^{-1}(3 - \ln(x+2)) \end{aligned}$$

2.2 Khảo sát với các độ chính xác khác nhau

Các độ chính xác được khảo sát là:

$$\text{tol} \in \{10^{-5}, 10^{-10}, 10^{-15}\}.$$

3 Phân tích chương trình

3.1 Khối 1: Import thư viện

```
1 import numpy as np
```

Listing 1: Import thư viện NumPy

Giải thích:

Khối lệnh này nạp thư viện NumPy để sử dụng các hàm toán học như:

- `np.exp(x)` cho hàm mũ e^x ,
- `np.cos(x)` cho hàm $\cos x$,
- `np.log(x)` cho hàm logarit tự nhiên,
- `np.cosh(x)` và `np.sinh(x)` cho các hàm hyperbol.
- `np.arccosh(x)` cho hàm ngược của $\cos$ hyperbolic.

3.2 Khối 2: Hàm thống kê số vòng lặp

```
1 def thongkesovonglap(i, tol, hamso, phuongphap, stats_file):  
2     with open(stats_file, "a", encoding="utf-8") as file:  
3         file.write(f"{hamso.__name__:>25s} {tol:12.3e} {phuongphap:>25s} {i:12d}\n")
```

Listing 2: Hàm ghi thống kê số vòng lặp

Giải thích:

Hàm này dùng để ghi kết quả tổng hợp vào tệp thống kê chung. Cụ thể:

- `i`: số vòng lặp đã thực hiện,
- `tol`: độ chính xác yêu cầu,
- `hamso`: tên hàm đang xét,
- `phuongphap`: tên phương pháp số,
- `stats_file`: tên tệp lưu bảng thống kê.

Lệnh `hamso.__name__` giúp lấy tên của hàm, ví dụ `hamso1`, `hamso2`, `hamso3`. Căn lề bằng các định dạng như `>15s` hay `:12.3e` làm cho bảng thống kê dễ đọc hơn.

3.3 Khối 3: Phương pháp chia đôi khoảng cách

```
1 def chiadoikhoangcach(a, b, tol, N, filename, hamso, stats_file):  
2     with open(f"{filename}_{hamso.__name__}_{tol}_chiadoikhoangcach.txt", "w", encoding="utf-8")  
3         as file:  
4         file.write(f"Giai phuong trinh f(x) = 0 bang PP chia doi khoang cach\n")  
5         file.write(f"Do chinh xac cua phep tinh la: {tol}\n")  
6         file.write(f"# * 100 + "\n")  
7         file.write(f"{i:>3s} {a:>12s} {b:>12s} {c:>25s} {f(c):>25s}\n")  
8         for i in range(0, N):  
9             c = (a+b)/2  
10            fa = hamso(a)  
11            fc = hamso(c)  
12            if fa*fc < 0:  
13                b = c  
14            else:  
15                a = c  
16            file.write(f"{i:3d} {a:12.6f} {b:12.6f} {c:25.16f} {fc:25.16e}\n")
```

```
16         if abs(fc)<tol:
17             break
18         if i == N - 1:
19             file.write(f"\n Không tìm được nghiệm sau {N} vòng lặp ")
20     thongkesovonglap(i+1,tol,hamso,'Chia đôi khoảng cách',stats_file)
```

Listing 3: Phương pháp chia đôi khoảng cách

Giải thích:

Phương pháp chia đôi yêu cầu ban đầu phải có đoạn $[a, b]$ sao cho $f(a)f(b) < 0$, tức là hàm đổi dấu trên đoạn đó. Mỗi vòng lặp:

- tính trung điểm $c = (a + b)/2$,
- xét dấu của $f(a)f(c)$,
- nếu $f(a)f(c) < 0$ thì nghiệm nằm trong $[a, c]$, nên gán $b = c$,
- ngược lại nghiệm nằm trong $[c, b]$, nên gán $a = c$.

Ưu điểm của phương pháp này là ổn định. Tuy nhiên tốc độ hội tụ chỉ là tuyến tính nên thường chậm hơn Newton và Secant.

3.4 Khối 4: Phương pháp điểm cố định

```
1 def diemcodinh(tol,p0,N,filename,hamso,stats_file):
2     with open(f"{filename}_{hamso.__name__}_{tol}_diemcodinh.txt", "w", encoding="utf-8") as
3         file:
4             file.write(f"Giai phuong trinh f(x) = 0 bang phuong phap diem co dinh\n")
5             file.write(f"Do chinh xac cua phep tinh la: {tol}\n")
6             file.write(f"# * 100 + "\n")
7             file.write(f"{i':>3s} {p':>25s} {g(p)':>20s}\n")
8             i = 1
9             for i in range(0,N):
10                 p = hamso(p0)
11                 file.write(f"{i:3d} {p0:25.16f} {p:25.16f}\n")
12                 if abs(p - p0) < tol:
13                     break
14                 p0 = p
15                 if i == N - 1:
16                     file.write("!" * 70 + "\n" + "!" * 70 + "\n")
17                     file.write(f"\n Không tìm được nghiệm sau {N} vòng lặp ")
18     thongkesovonglap(i+1,tol,hamso,'Diem co dinh',stats_file)
```

Listing 4: Phương pháp điểm cố định

Giải thích:

Phương pháp điểm cố định biến đổi phương trình về dạng

$$x = g(x).$$

Khi đó, từ giá trị ban đầu x_0 , ta xây dựng vòng lặp

$$x_{n+1} = g(x_n).$$

Phương pháp này chỉ hội tụ khi hàm lặp $g(x)$ cần thỏa điều kiện

$$|g'(x)| < 1$$

3.5 Khối 5: Phương pháp Newton–Raphson

```
1 def newton_raphson(tol, p0, N, filename, hamso, daoham, stats_file):
2     with open(f"{filename}_{hamso.__name__}_{tol}_newton_raphson.txt", "w", encoding="utf-8") as file:
3         file.write(f"Giai phuong trinh f(x) = 0 bang phuong phap Newton Raphson\n")
4         file.write(f"Do chinh xac cua phep tinh la: {tol}\n")
5         file.write(f"# * 100 + "\n")
6         file.write(f"{ 'i':>3s} { 'p':>25s} { 'f(p)':>25s}\n")
7         i = 1
8         for i in range(0, N):
9             if daoham(p0) == 0:
10                 file.write("!" * 70 + "\n" + "!" * 70 + "\n")
11                 file.write(f"\n Khong tim duoc nghiem do dao ham = 0'")
12                 break
13             p = p0 - hamso(p0)/daoham(p0)
14             file.write(f"{i:3d} {p:25.16f} {hamso(p):25.16f}\n")
15             if abs(p - p0) < tol:
16                 break
17             p0 = p
18             if i == N - 1:
19                 file.write("!" * 70 + "\n" + "!" * 70 + "\n")
20                 file.write(f"\n Khong tim duoc nghiem sau {N} vong lap'")
21             thongkesovonglap(i+1, tol, hamso, 'Newton-Raphson', stats_file)
```

Listing 5: Phương pháp Newton–Raphson

Giải thích:

Newton–Raphson sử dụng công thức lặp:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Code trên thực hiện vòng lặp này:

- kiểm tra $f'(x_n) \neq 0$,
- tính giá trị mới p ,
- ghi lại p và $f(p)$,
- dừng khi $|p - p_0| < \text{tol}$.

3.6 Khối 6: Phương pháp Secant

```
1 def phuongphap_secant(tol, p0, p1, N, filename, hamso, stats_file):
2     with open(f"{filename}_{hamso.__name__}_{tol}_secant.txt", "w", encoding="utf-8") as file:
3         file.write(f"Giai phuong trinh f(x) = 0 bang phuong phap Secant\n")
4         file.write(f"Do chinh xac cua phep tinh la: {tol}\n")
5         file.write(f"# * 100 + "\n")
6         file.write(f"{ 'i':>3s} { 'p':>25s} { 'f(p)':>25s}\n")
7         i = 1
8         for i in range(0, N):
9             p = p1 - (hamso(p1)*(p1-p0))/(hamso(p1)-hamso(p0))
10            file.write(f"{i:3d} {p:25.16f} {hamso(p):25.16f}\n")
11            if abs(p - p1) < tol:
12                break
13            p0 = p1
14            p1 = p
15            if i == N - 1:
16                file.write("!" * 70 + "\n" + "!" * 70 + "\n")
17                file.write(f"\n Khong tim duoc nghiem sau {N} vong lap'")
18            thongkesovonglap(i+1, tol, hamso, 'Secant', stats_file)
```

Listing 6: Cài đặt phương pháp Secant

Giải thích:

Secant có công thức:

$$x_{n+1} = x_n - \frac{f(x_n) \times (x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}.$$

Phương pháp này có thể xem là phiên bản gần đúng của Newton, trong đó đạo hàm được thay bằng sai phân hữu hạn. Vì vậy:

- không cần tính đạo hàm,
- thường hội tụ nhanh hơn chia đôi,
- tốc độ hội tụ thường chậm hơn Newton nhưng nhanh hơn bậc tuyến tính.

Một lưu ý kỹ thuật là nên kiểm tra mẫu số hamso(p1)-hamso(p0) $\neq 0$ để tránh lỗi chia cho 0.

3.7 Khối 7: Định nghĩa các hàm số và đạo hàm

```
1 def hamso1(x):
2     y = np.exp(-x) - x**2
3     return y
4
5 def hamso1_daoham(x):
6     y = -np.exp(-x) - 2*x
7     return y
8
9 def hamso2(x):
10    y = x**3 - np.cos(x)
11    return y
12
13 def hamso2_daoham(x):
14    y = 3*x**2 + np.sin(x)
15    return y
16
17 def hamso3(x):
18    y = np.log(x+2) + np.cosh(x) - 3
19    return y
20
21 def hamso3_daoham(x):
22    y = 1/(x+2) + np.sinh(x)
23    return y
24
25 def hamso1_ppdiemcodinh(x):
26    g = np.exp(-x/2)
27    return g
28 def hamso2_ppdiemcodinh(x):
29    g = (np.cos(x))**(1/3)
30    return g
31 def hamso3_ppdiemcodinh(x):
32    g = np.arccosh(3-np.log(x+2))
33    return g
```

Listing 7: Các hàm số khảo sát và đạo hàm tương ứng

Giải thích:

Khối này định nghĩa ba bài toán cụ thể và các đạo hàm tương ứng để dùng cho phương pháp Newton–Raphson. Việc tách riêng hàm và đạo hàm làm cho chương trình có cấu trúc rõ ràng, dễ mở rộng nếu muốn thêm hàm số mới.

3.8 Khối 8: Thiết lập tham số chạy chương trình

```
1 # Phương pháp chia đôi
2 a1 = -20
3 b1 = 20
4
5 # Phương pháp Newton Raphson, điểm cố định
6 a2 = 5
7
8 # Phương pháp secant
9 a3 = -1
10 b3 = 1
11
12 N = 100
13 filename = 'BTVNlan1'
14 filethongke = 'Thongkesovonglap'
15
16 with open(filethongke, "w", encoding="utf-8") as file:
17     file.write("Thong ke so vong lap theo tung phuong phap va do chinh xac cho tung loai ham
18         so\n")
19     file.write("#" * 100 + "\n")
20     file.write(f"{'Ham so':>25s} {'Do chinh xac':>12s} {'Phuong phap':>25s} {'So vong
21         lap':>12s}\n")
```

Listing 8: Khai báo tham số và chuẩn bị file thống kê

Giải thích:

Các biến này được dùng để:

- xác định khoảng đầu $[a, b]$,
- đặt số vòng lặp tối đa $N = 100$,
- chọn tên chung cho các tệp kết quả,
- tạo tệp thống kê ban đầu với tiêu đề cột.

Tệp Thongkesovonglap sẽ đóng vai trò như một bảng tổng hợp toàn bộ số vòng lặp của mọi phương pháp.

3.9 Khối 9: Tạo danh sách hàm số và độ chính xác

```
1 danhsachhamso = [hamso1, hamso2, hamso3]
2 danhsachhamso_daoham = [hamso1_daoham, hamso2_daoham, hamso3_daoham]
3 danhsachhamso_diemcodinh = [hamso1_ppdiemcodinh, hamso2_ppdiemcodinh, hamso3_ppdiemcodinh]
4 danhsach_tol = [1e-5, 1e-10, 1e-15]
```

Listing 9: Danh sách hàm số, đạo hàm và ngưỡng sai số

Giải thích:

Các danh sách này cho phép chương trình chạy tự động trên nhiều trường hợp khác nhau mà không cần viết lặp thủ công. Đây là cách tổ chức tốt vì:

- gọn mã nguồn,
- dễ thay đổi bộ dữ liệu khảo sát,
- thuận tiện cho việc mở rộng báo cáo thực nghiệm.

3.10 Khối 10: Vòng lặp chính thực hiện tính toán

```
1 for x in range(len(danhsachhamso)):
2     for y in range(len(danhsach_tol)):
3         chiadoikhoangcach(a1, b1, danhsach_tol[y], N, filename, danhsachhamso[x], filethongke)
4         diemcodinh(danhsach_tol[y], a2, N, filename, danhsachhamso_diemcodinh[x], filethongke)
5
```

```
6 newton_raphson(danh_sach_tol[y], a2, N, filename, danh_sach_hamso[x],  
7  
8 danh_sach_hamso_daoham[x], file_thong_ke)  
9 phương_phap_secant(danh_sach_tol[y], a3, b3, N, filename, danh_sach_hamso[x], file_thong_ke)
```

Listing 10: Vòng lặp chạy toàn bộ các phương pháp

Giải thích:

Đây là phần chạy của toàn bộ chương trình. Với mỗi hàm số và mỗi giá trị tol, chương trình sẽ lần lượt chạy bốn phương pháp:

1. chia đôi khoảng cách,
2. điểm cố định,
3. Newton–Raphson,
4. Secant.

Mỗi lần chạy sẽ sinh ra:

- một tệp riêng ghi chi tiết từng bước lặp,
- một dòng trong bảng thống kê tổng.

4 So sánh các phương pháp đã trình bày

4.1 Ảnh hưởng của độ chính xác tol

Khi giảm tol từ 10^{-5} xuống 10^{-10} rồi 10^{-15} , số vòng lặp cần thiết thường tăng lên. Tuy nhiên mức độ tăng là khác nhau:

- **Phương pháp chia đôi khoảng cách:** số vòng lặp tăng khá đều vì sai số giảm theo cấp số nhân với cơ số 1/2. Đây là phương pháp chậm nhất trong nhóm các phương pháp hội tụ chắc chắn.
- **Newton–Raphson:** số vòng lặp thường tăng rất ít khi yêu cầu chính xác cao hơn, do gần nghiệm thì số chữ số đúng tăng rất nhanh sau mỗi bước lặp.
- **Secant:** tốc độ kém Newton một chút nhưng vẫn nhanh hơn đáng kể so với chia đôi.
- **Điểm cố định:** phụ thuộc vào $g(x)$, nếu chọn hàm $g(x)$ tốt thì có thể hội tụ ổn; nếu chọn không phù hợp thì hội tụ rất chậm hoặc phân kỳ.

4.2 Kết quả chạy

Sau khi chạy chương trình số vòng lặp theo từng hàm thử, độ chính xác và các phương pháp như sau:

Hàm số	Độ chính xác	Phương pháp	Số vòng lặp
hamso1	10^{-5}	Chia đôi khoảng cách	20
		Điểm cố định	13
		Newton–Raphson	6
		Secant	6
	10^{-10}	Chia đôi khoảng cách	39
		Điểm cố định	24
		Newton–Raphson	7
		Secant	7
	10^{-15}	Chia đôi khoảng cách	48
		Điểm cố định	35
		Newton–Raphson	8
		Secant	8
hamso2	10^{-5}	Chia đôi khoảng cách	23
		Điểm cố định	12
		Newton–Raphson	8
		Secant	7
	10^{-10}	Chia đôi khoảng cách	39
		Điểm cố định	22
		Newton–Raphson	9
		Secant	8
	10^{-15}	Chia đôi khoảng cách	54
		Điểm cố định	33
		Newton–Raphson	10
		Secant	9
hamso3	10^{-5}	Chia đôi khoảng cách	21
		Điểm cố định	10
		Newton–Raphson	8
		Secant	6
	10^{-10}	Chia đôi khoảng cách	39
		Điểm cố định	17
		Newton–Raphson	9
		Secant	7
	10^{-15}	Chia đôi khoảng cách	53
		Điểm cố định	24
		Newton–Raphson	10
		Secant	8

4.3 Kết luận so sánh

Nếu bỏ qua các trường hợp phân kỳ và xét trong điều kiện ban đầu thuận lợi, thứ tự hội tụ nhanh dần thường là:

Chia đôi < Điểm cố định < Secant < Newton–Raphson.